Оглавление

Лабораторная работа №1. Применение методов интуитивного тестирования и отладки при анализе исходного кода. 3

Цель работы: 3

Объект тестирования: 3

Задание: 3

Выявление потенциальных сценариев отказа 4

Журнал дефектов 5

Решение: 6

Этап разработки: 7

Основные тесты. 9

Предотвращение ошибок. 11

Проведение юнит-тестов. 13

Пояснение касаемо данных объекта: 14

Результаты юнит-тестов. 14

Подробные данные 15

Вывод 20

Ссылка на проект на Github: 20

Лабораторная работа №2. Модульное тестирование в программной среде 21

Цель работы: 21

Задачи: 21

Выявление потенциальных сценариев отказа 21

Журнал дефектов 22

Решение: 23

Этап разработки. 23

Результаты модульных тестов. 27

Предотвращение ошибок 27

Вывод 29

Ссылка на проект на Github 29

Лабораторная работа №1. Применение методов интуитивного тестирования и отладки при анализе исходного кода.

Цель работы:

Приобретение практических навыков выявления дефектов обеспечения программного методами интуитивного тестирования (предугадывания ошибок) и отладки исходного кода на языке высокого уровня.

Объект тестирования:

Приложение, предназначенное для обработки одномерного массива целых чисел.

Задание:

Определите сколько обменов сделает алгоритм пузырьковой сортировки по возрастанию для данного массива.

Формат входных данных :

На первой строке дано целое число n($1 \leq n \leq 1000$) – количество элементов в массиве. На второй строке – сам массив. Гарантируется, что все элементы массива – различные целые числа, не превышающие по модулю 10^9.

Формат выходных данных:

Выведите одно число – количество обменов пузырьковой сортировки.

Задание на лабораторную работу: Выполнить отладку программы. Провести тестирование с исходными данными

//Вход 3 1 2 3 Ответ 0 1 2 3

//Вход 2 2 1 Ответ 1 1 2

//Вход 4 1 5 3 Ответ 3 1 3 4 5

**на оценку отлично

Дополнительно автоматизировать тестирование за счет прогона тестов из файла в количестве 10 тестовых случаев

Вывести на экран результат тестирования в формате Тест 1 Результат да или нет

Выявление потенциальных сценариев отказа

В задаче имеется некоторое количество переменных, задаваемых пользователем. Нужно учитывать, что такие значения могут содержать ошибки, или намеренно пытаться «сломать» код. Поэтому нужно избавиться от всех возможных ошибок на этапе валидации.

Во-первых, значение n – количество элементов в массиве. Это значение задаётся пользователем. Могут возникнуть следующие разновидности ошибок:

- Что если введено не число? – Необходимо ввести проверку на число.
- Раз число должно отражать количество – оно не может отрицательным. Позволять ему быть нулём тоже имеет мало смысла, так как вряд ли получиться успешно сортировать массив нулевой длины, поэтому необходимо установить минимум в единицу.
- По вышеописанной причине так же необходимо ввести проверку на целочисленность.

Во-вторых, что касаемо массива, передаваемого пользователем:

- Необходимо корректно превращать отправленную пользователем строку с числами в массив – в моём случае разделение чисел будет осуществляться пробелом

- В соответствии с задачей так же необходимо убедиться, что массив состоит из целых чисел – перед сортировкой они будут округляться до целых
- По вышеописанной причине во время процесса валидации будет логичным заменить все запятые на точки, так как если пользователь из России захочет написать нецелое число, ему будет привычнее написать запятую – чтобы это не ломало код, её необходимо заменить на точку, а после этого округлить число

Также был добавлен **функционал изменения максимального модуля числа**, для корректной работы которого тоже есть несколько условий:

- Оно должно быть корректно преобразовано в число.
- При этом имеет смысл позволить пользователю использовать символ ^ как привычный символ возведения в степень, потому как в задаче изначально описано значение 10^9 и писать вручную 1000000000 было бы неудобно. А потому необходимо каким-либо образом реализовать механизм вычисления степени с помощью этого.

Журнал дефектов

Таблица 1 – Журнал дефектов

№	Входные данные	Фактический результат	Ожидаемый результат	Локали--зация в коде	Причина возникновения
1	Модуль = не число	Падение программы	Вывод ошибки	38	В поле «Модуль» передано не число
2	N=не число	Падение программы	Вывод ошибки	46	В поле передано не число
3	N=3, array= [1, 2, 3, 4]	Программа продолжает работу	Вывод ошибки	137	В массив передано больше чисел, чем указано в n
4	Array = [‘string’]	Падение программы	Вывод ошибки	71	В массив передано не число

Продолжение Таблицы 1

№	Входные данные	Фактический результат	Ожидаемый результат	Локали--зация в коде	Причина возникновения
5	N=-5	Программа продолжает работу	Вывод ошибки	135	Передано количество меньше 1
6	Модуль=10 Array=[12]	Программа продолжает работу	Вывод ошибки	70	Передано число, превышающее макс. модуль
7	Модуль = -10	Падение программы	Число берётся по модулю	66	В модуль передано отрицательное число
8		Падение программы	Вывод результат во встроенную консоль	75	Отсутствует поле вывод ошибок
9	Array= ”1,5 2,5”	Падение программы	Замена запятой точкой	55	Запятая превращает значение в NaN – не число
10	Array=[1.5]	Программа продолжает работу	Округление нецелых чисел	58	Программа принимает любые числа, но условие требует лишь целые
11	Array=[1 2]	Программа сортирует в том числе пустые элементы (пробелы)	Сортировка лишь чисел	58	Разделение строки на массив не учитывает много пробелов подряд

Решение:

Для решения задачи был использован язык JavaScript, а также HTML и CSS для более наглядного отображения в интерфейсе.

Этап разработки:

Функция пузырьковой сортировки была адаптирована под выбранный ЯП:

```
function bubbleSort(array, returnMassive = false, size = array.length) {
    let newArray = array;
    let swap_counter = 0;
    for (let i = 1; i < size; ++i) {
        for (let j = 1; j <= size - i; ++j) {
            if (newArray[j - 1] > newArray[j]) {
                // swap
                let temp1 = newArray[j-1];
                let temp2 = newArray[j];
                newArray[j] = temp1;
                newArray[j-1] = temp2;
                // counter
                swap_counter++;
            }
        }
    }

    return returnMassive ? newArray : swap_counter;
}
```

Рисунок 1 – Функция сортировки пузырьком

Функция была изменена так, чтобы при необходимости выводить не только количество перестановок, но и итоговый отсортированный массив.

После небольших тестов работоспособности был создан интерфейс для возможности пользователю вводить значения удобнее:

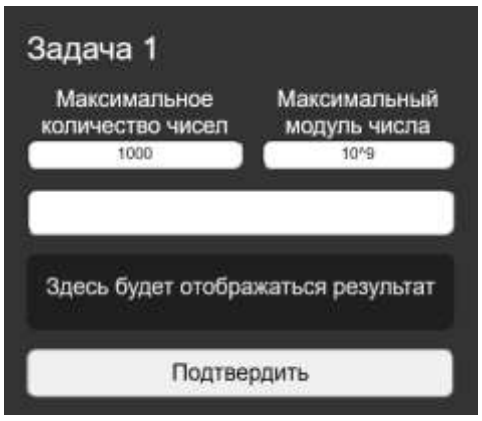


Рисунок 2 – Интерфейс для тестов

Для валидации значений первых двух полей была создана следующая функция:



Рисунок 3 – Функция обработки значений

Эта функция для каждого из полей определяет, ввёл ли пользователь число. Если нет, то в случае с первым полем, от пользователя потребуется ввести число и ничто другое, в случае со вторым полем, допускается использование символа ^ для обозначения степени. В обоих случаях также полученные числа дополнительно ограничиваются строго укладывается в диапазон **n = (1 ≤ n ≤ 1000)** – количество элементов в массиве, а также различные целые числа, не превышающие по модулю **10^9**. Эти ограничения устанавливаются как аргументы функции при её объявлении, по стандарту устанавливаются вышеописанные.

Ниже показано отображение ошибок:

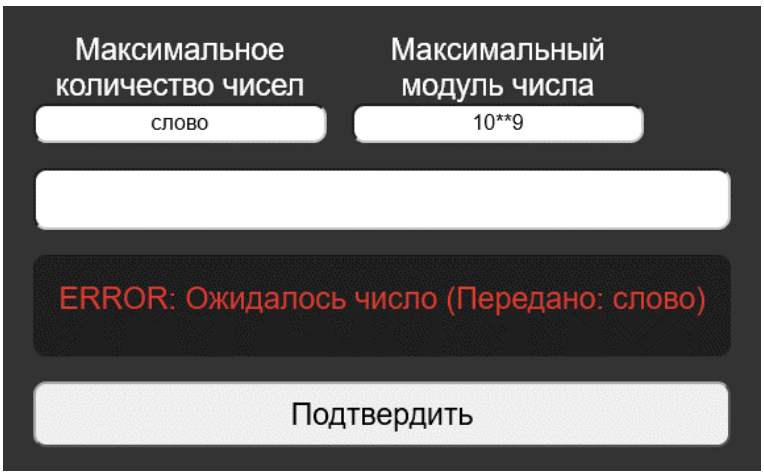


Рисунок 4 – Ошибка строки

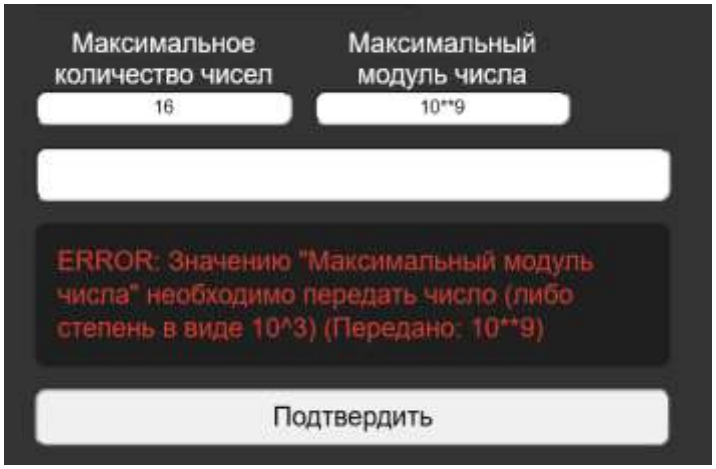


Рисунок 5 – Ошибка неверной строки

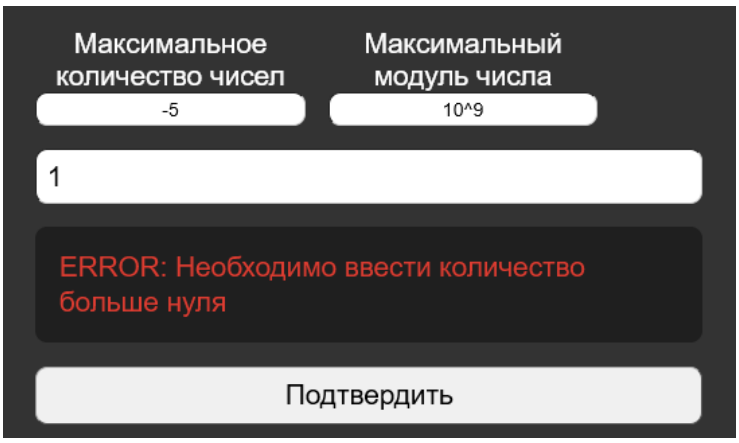


Рисунок 6 – Ошибка неверного количества чисел

Основные тесты.

После создания некоторых необходимых подфункций были проведены тесты работы алгоритма на основе данных в задании массивов:

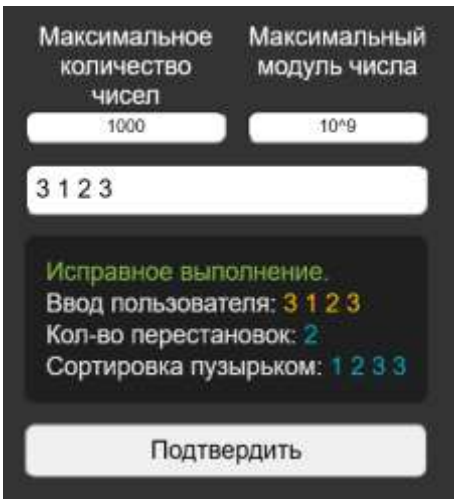


Рисунок 7 – Проверка на массиве 3 1 2 3

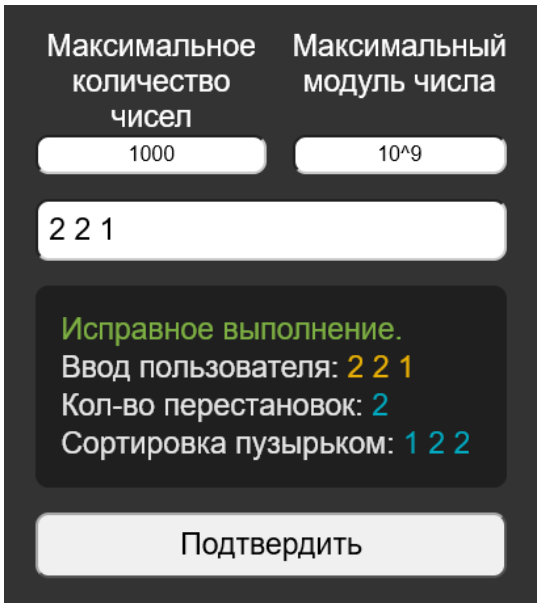


Рисунок 8 – Проверка на массиве 2 2 1

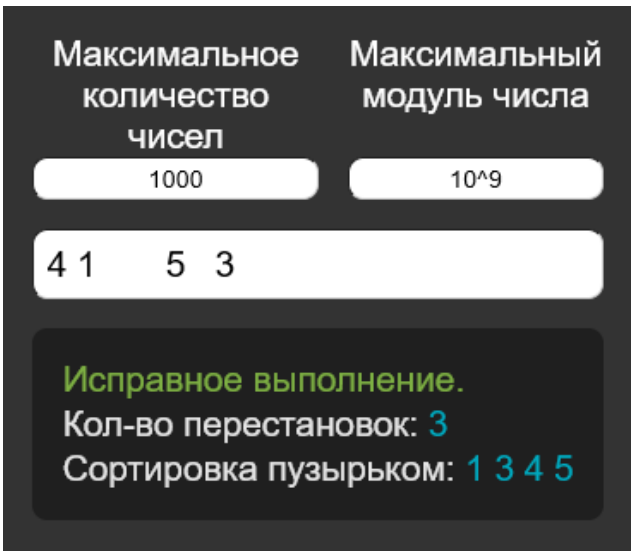


Рисунок 9 – Проверка на массиве 4 1 5 3

Предотвращение ошибок.

На Рисунке 9 также отображено, что исправно регулируется количество пробелов, отправленное пользователем, и это не ломает логику алгоритма. Различные знаки препинания (кроме точек и запятых) или текст не пропускается и выдаёт ошибку:

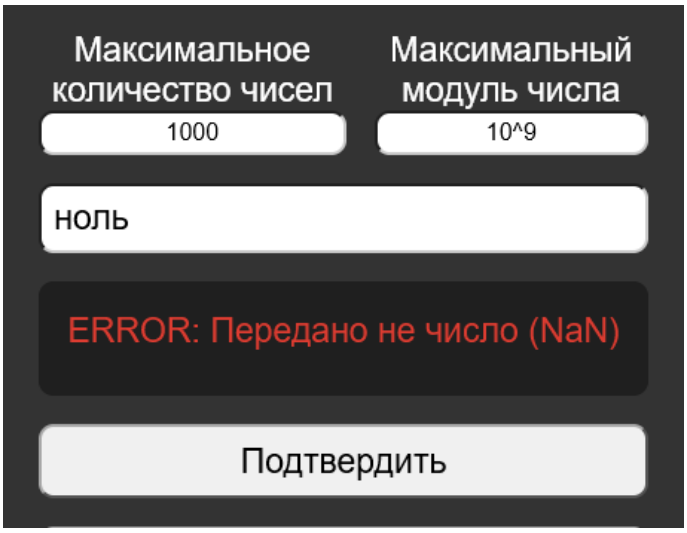


Рисунок 10 – Ошибка - не число

Точки с запятыми допускаются, но так как по условию массив должен состоять из целых чисел, они округляются до целых:

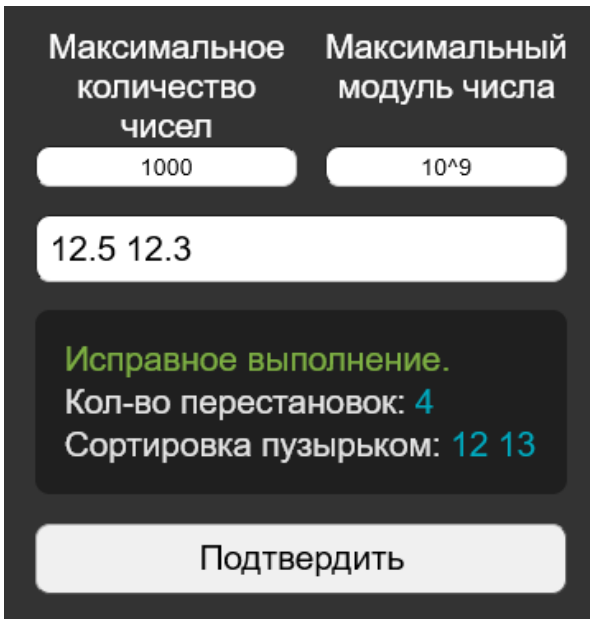


Рисунок 11 – Округление чисел

Также была введена валидация каждого числа на соответствие модулю и не превышает ли переданное количество чисел максимум:

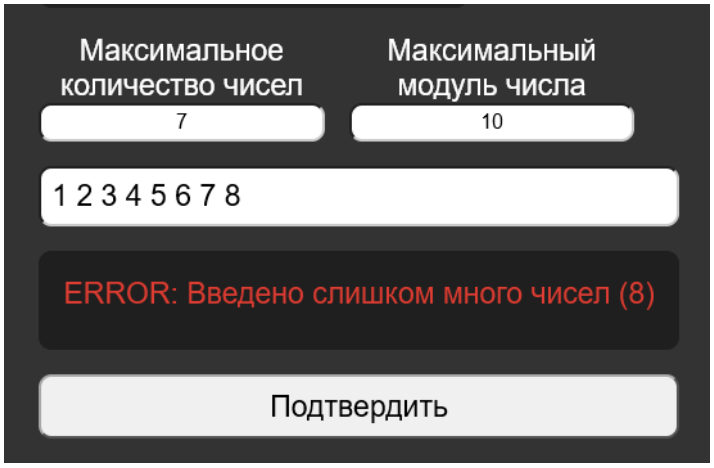


Рисунок 12 – Слишком много чисел

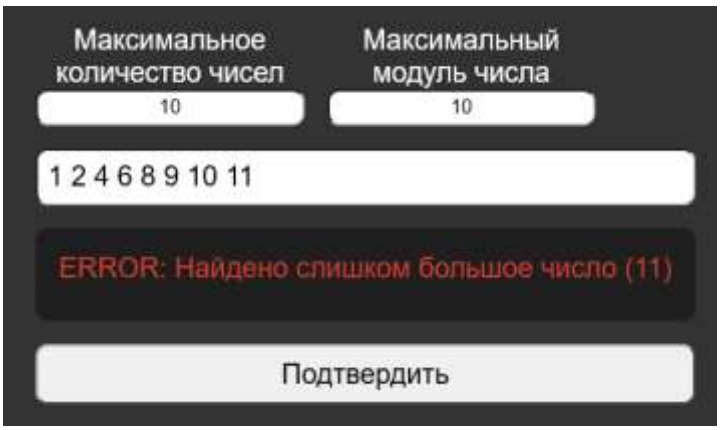


Рисунок 13 – Непозволительно большой модуль

Проведение юнит-тестов.

В соответствии с заданием необходимо провести 10 заранее заготовленных тестов, на основе которых будет наглядно видно результат работы.

В JavaScript существует встроенный метод сортировки массивов **.sort()**, в который, если не передавать никаких параметров, просто сортирует числа по возрастанию. Этот метод будет для нас отличным ориентиром того, сработала ли наша рукописная сортировка пузырьком.

Проведение тестов будет проводиться по следующему механизму:

- Подготавливаем различные граничные значения, которые подозреваются в потенциальном вызове ошибки
- Устанавливаем различные значения n, значения модуля, и массивы
- Заранее указываем, ожидается ли что тест провалится. Это позволит определить границы работоспособности системы методом исключения – на каком этапе программа падает, а не какие она проходит.

Таким образом был подготовлен объект со следующими тестами:



```
150
151     let unitTestSet = new Set([
152         {number:1, array:'3 1 2 3', n:1000, module:10**9, expected: true},
153         {number:2, array:'2 2 1', n:1000, module:10**9, expected: true},
154         {number:3, array:'4 1 5 3', n:1000, module:10**9, expected: true},
155         {number:4, array:'5 4 3 2 1 0', n: 6, module:10**9, expected: true},
156         {number:5, array:'5 4 3 2 1 0', n: 5, module:10**9, expected:false},
157         {number:6, array:'26 09 1', n: 7, module:10**9, expected:true},
158         {number:7, array:'0 4 9 8 10', n: 8, module:10, expected:true},
159         {number:8, array:'0 4 9 8 10', n: 8, module:9, expected:false},
160         {number:9, array:'0 0 0 0', n: 5, module:50, expected:true},
161         {number:10, array:'0 1 2 3 4 5', n: 5, module:1, expected:false}
162     ]);
```

Рисунок 14 – Объект с данными для юнит-тестов

Пояснение касаемо данных объекта:

number – порядковый номер,

array – сам сортируемый массив,

n – максимальное количество чисел в массиве,

module – максимальный модуль числа,

expected – ожидаемый результат, провал теста или успех – false или true

Результаты юнит-тестов.

В пользовательский интерфейс также была добавлена соответствующая кнопка «Провести юнит-тесты», нажатие на которую создаёт 10 карточек с результатами:



Рисунок 15 – Результаты юнит-тестов

Как видно из результатов, функция справляется с сортировкой по возрастанию точно так же хорошо, как и встроенный метод **.sort()**. Среди десяти тестов все выходные значения совпали. Это показывает, что функция работает исправно с любыми массивами.

Подробные данные.

Однако это не показывает, где она спотыкается. А потому для возможности более дотошного просмотра выходных данных и дебаггинга был добавлен тумблер, переключающий отображение дополнительных настроек.

Вот пример того, что выводится в окно с результатами при включенном флаге «Выводить расширенные данные»:

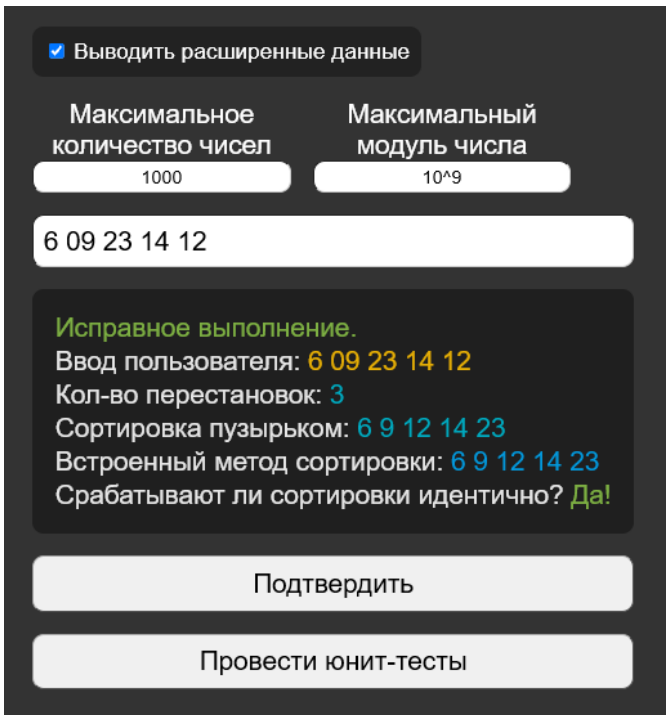


Рисунок 16 – Подробные результаты

Здесь помимо количества перестановок отображается массив, отсортированный встроенной функцией, и короткое пояснение, схожи ли массивы.

На данный момент, судя по моим бесконечным тестам, невозможно получить ответ нет, но если через код намерено вызвать ошибку, это отобразится в результатах:

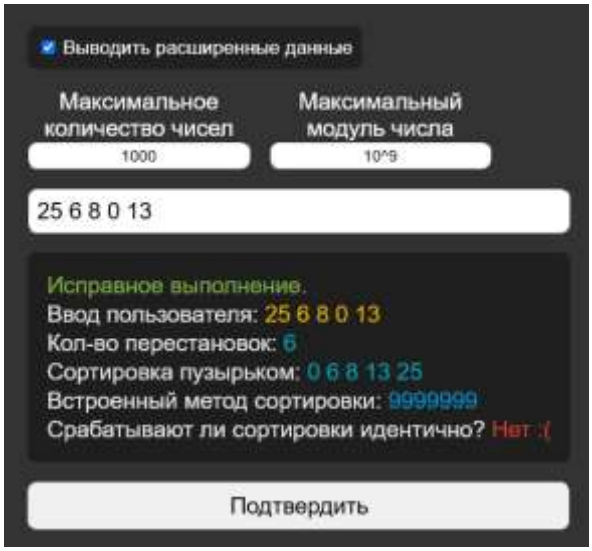


Рисунок 17 – Намеренно созданный баг

Также эта настройка, отображающая подробные результаты влияет на то как отображаются юнит-тесты. Теперь все те данные, которые в них закладывались объектом **UnitTestSet** могут быть замечены.

Вот пример первых трёх массивов (тех, что были обозначены в примере):

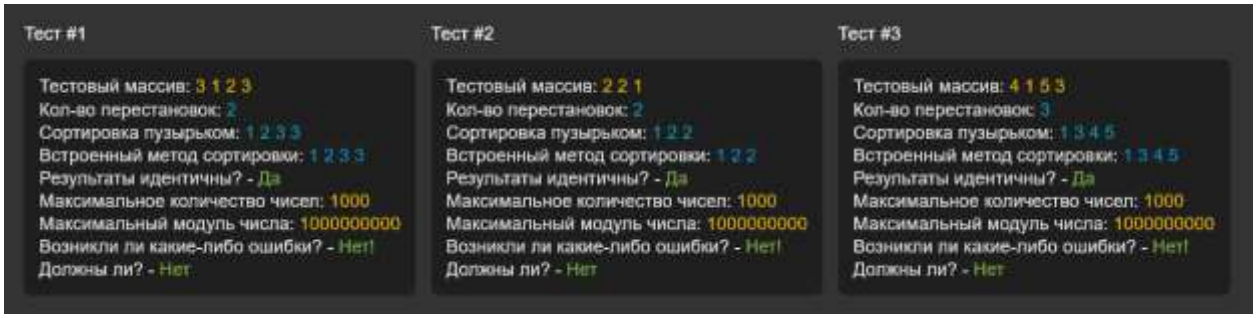


Рисунок 18 – Расширенные результаты первых трёх юнит-тестов

Ранее было заявлено, что какие-то тесты должны были намеренно пройти неудачно, для выявления ошибки методом исключения и убеждения в том, что обработка этих ошибок работает исправно.

Вот один из примеров, когда ошибка ожидалась, она произошла, и в результатах вывелось, что конкретно произошло:

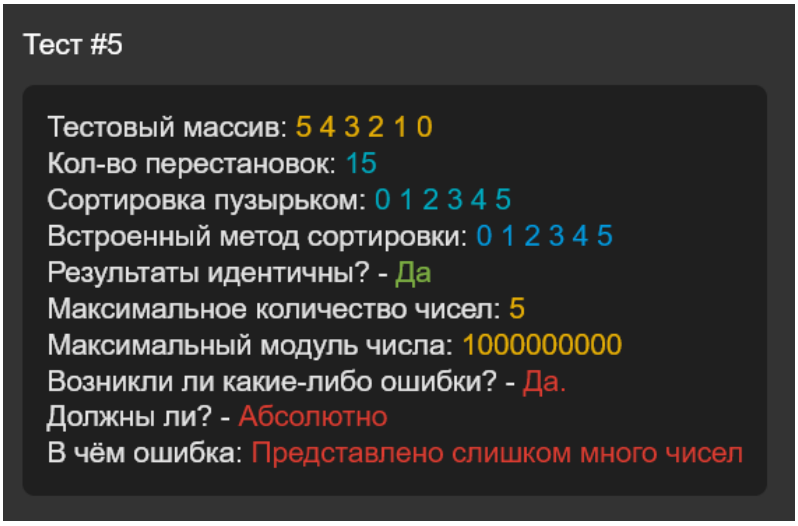


Рисунок 19 – Ошибочный тест №5

В тесте №5 (см. Рисунок 19) значение n (максимальное кол-во чисел) установлено на 5, а в массиве передано 6 чисел. И, хотя, массив исправно отсортировался, ошибка не прошла незамеченной.

Параметр **expected** (ожидаемое) как раз-таки отвечал, за то, ждём ли мы ошибки. Обработчик ошибок поймал одну, и в поле **«Возникли ли какие-либо ошибки?»** вписал **«Да.»**. А затем, в соответствии с параметром **expected**, в поле **«Должны ли?»** (имеется ввиду «должны ли были возникнуть ошибки») вписалось слово **«Абсолютно»**, что подразумевает согласие – мы ждали ошибку, и дождались. Затем, обработчик добавляет лишь в случае, если ошибка возникла, строчку, где описывается что сломалось. В данном примере в массив было передано слишком много чисел.

Таким образом срабатывают все тесты с различными параметрами.

Результаты юнит-тестов с расширенными параметрами результатов:

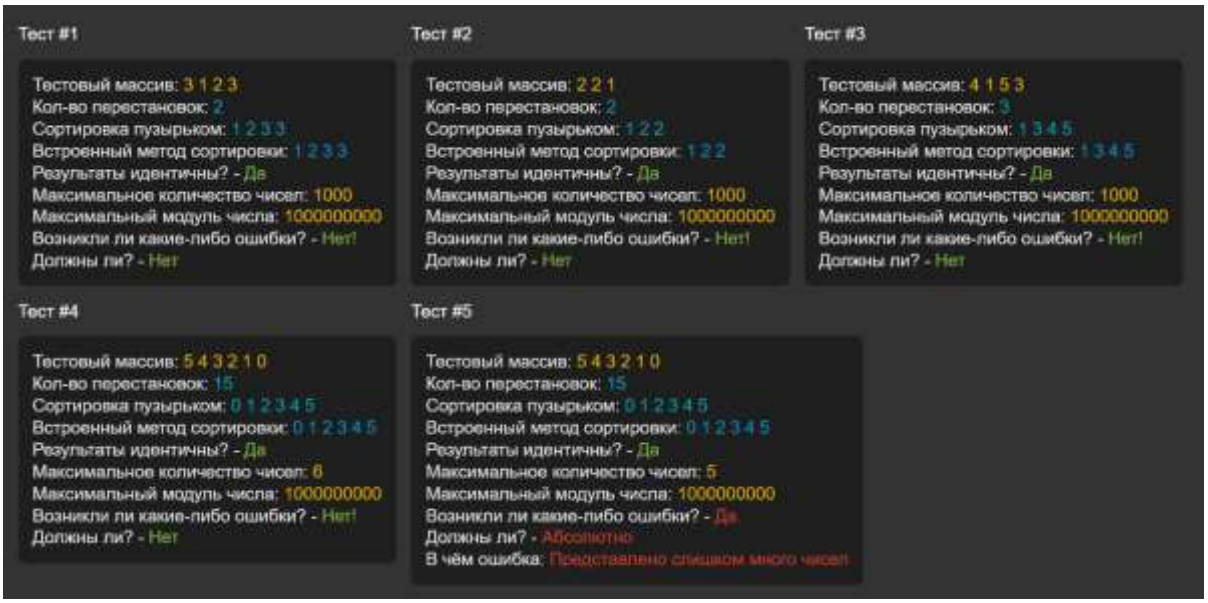


Рисунок 20 – Тесты ½



Рисунок 21 – Тесты 2/2

Как видно из результатов тестов, ошибки возникли лишь там, где мы их обрабатывали, и нигде больше.

Точно так же обработчик срабатывает в интерфейсе, где пользователю позволяется ввести входные данные.

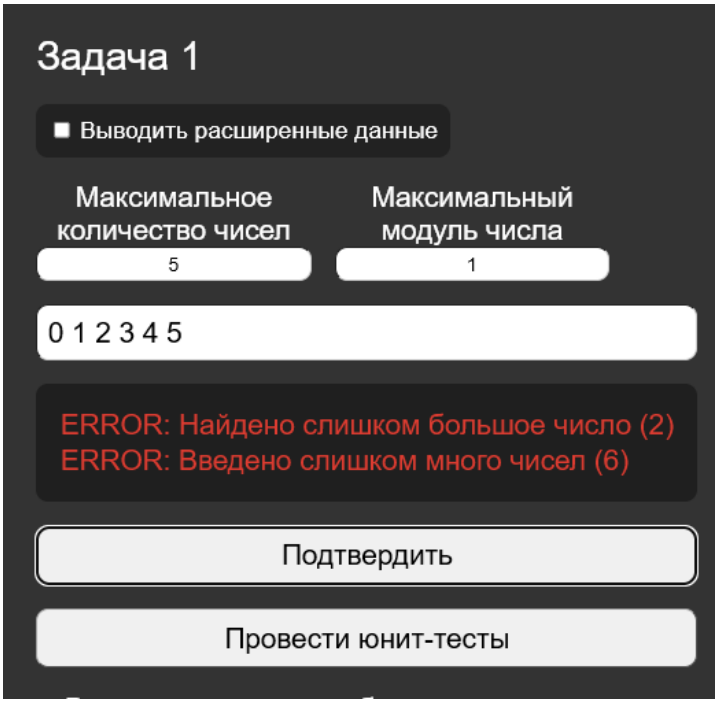


Рисунок 22 – Ошибки

Вывод

Таким образом был разработан алгоритм сортировки пузырьком с различными настраевыми параметрами, которые как видно из результатов юнит-тестов, исправно проходят валидацию.

Ссылка на проект на Github:

Исходный код на Github

Лабораторная работа №2. Модульное тестирование в программной среде.

Цель работы:

Получить навыки автоматизации модульного тестирования средствами среды разработки.

Задачи:

1. Разработать модуль тестирования в рамках программного решения
2. Разработать тестируемый класс

2.1. Разработать метод вычисления суммы двух чисел

2.2. Разработать метод вычисления произведения двух чисел

2.3. Разработать метод вычисления факториала

3. Проверить работоспособность модуля тестирования

Выявление потенциальных сценариев отказа

Так как наш класс должен работать с числами и операциями над ними, необходимо корректно реализовать *валидацию* данных.

Описанные выше методы *вычисления суммы, произведения и факториала* должны в качестве своих аргументов принимать лишь числа, либо строки, которые можно перевести в *числовой тип данных*.

Журнал дефектов

Таблица 2

№	Входные данные	Фактический результат	Ожидаемый результат	Локали--зация в коде	Причина возникновения
1	Sum(“1”, false)	Падение программы с ошибкой	Попытка преобразовать данные в число, вывод ошибки в случае неудачи	3	Методу вычисления суммы переданы некорректные данные
2	multiply(“1”, false)	Падение программы с ошибкой	Попытка преобразовать данные в число, вывод ошибки в случае неудачи	6	Методу вычисления произведения переданы некорректные данные
3	Factorial(null)	Падение программы с ошибкой	Попытка преобразовать данные в число, вывод ошибки в случае неудачи	13	Методу вычисления факториала передано некорректное значение
4	Fetch(“wrong API”)	Падение программы с ошибкой	Вывод ошибки	32	При запросе API передан неверный URL
5	Factorial(<1)	Вывод ошибки	Результат = 1	87	Факториал числа меньше единицы не вычисляется
6	n>max, m<min	Продолжение выполнения программы или ошибка	Установление значения в соответствии с ограничениями и вычисление	168	Преодоление допустимых установленных пределов аргументов
7	Input = NaN	Падение программы с ошибкой	Вывод ошибки	170	Передано не число

Продолжение Таблицы 2

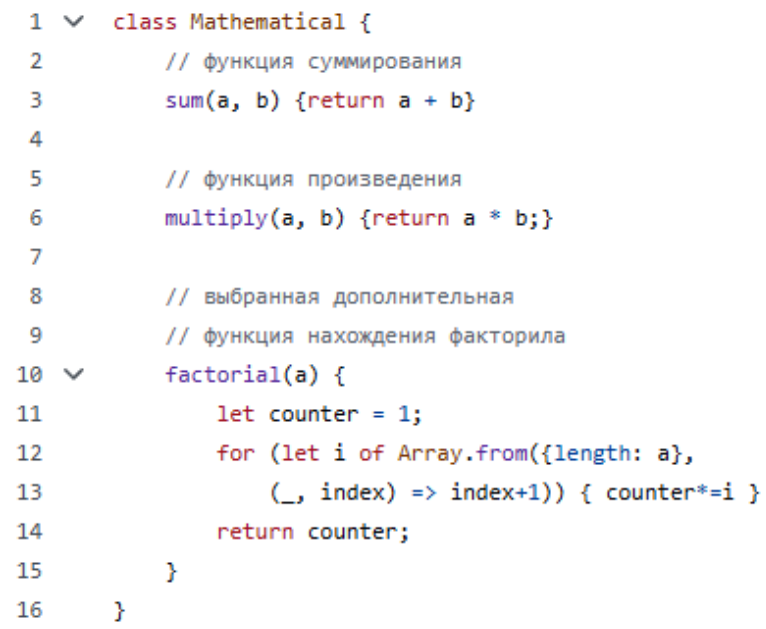
№	Входные данные	Фактический результат	Ожидаемый результат	Локали--зация в коде	Причина возникновения
8	Input = null	Падение программы с ошибкой	Вывод ошибки	167	Передано неопределённое или пустое значение
9	Max=null Min=null	Падение программы с ошибкой	Установка дефолтного значения	185	Если min и\или max не нашлось
10	Inputs.size > 2	Падение программы с ошибкой	Обработка и вывод ошибки	212	Количество инпутов слишком велико (максимум 2 – для суммы и произведения)

Решение:

Для решения задачи был использован ЯП JavaScript, а также HTML и CSS для реализации инпутов в интерфейсе и более наглядного отображения результативных данных.

Этап разработки.

Для начала был создан класс с описанными заданием методами:



```
class Mathematical {
    // функция суммирования
    sum(a, b) {return a + b}

    // функция произведения
    multiply(a, b) {return a * b;}

    // выбранная дополнительная
    // функция нахождения факторила
    factorial(a) {
        let counter = 1;
        for (let i of Array.from({length: a},
            (_, index) => index+1)) { counter*=i }
        return counter;
    }
}
```

Рисунок 23 – Класс с тремя методами

Для проведения достоверного Unit-тестирования было принято решение использовать официальный API встроенной библиотеки Math.js. Напрямую она не имеет в себе метода нахождения факториала, однако при обращении через POST-запрос, API позволяет ввести выражение в любом строковом виде и оно будет вычислено. Таким образом если обратиться по адресу *api.mathjs.org/v4/?expr=5!* на странице будет отображено 120 (что является результатом выражения 5!). Как итог, для проверки корректной работоспособности мы будем обращаться с определёнными выражениями и если их результаты будут совпадать с теми, которые были вычислены нашей функцией, это будет означать что всё работает и всё написано правильно.

Для обращения к API была написана следующая функция, которая внутри себе имеет все необходимые предотвращения ошибок

```
async function fetchRequest(expressionString, presicion=14) {
    const requestOptions = {
        method: "POST",
        headers: {"Content-Type": "application/json"},
        body: JSON.stringify({expr: expressionString})
    };

    const url = "https://api.mathjs.org/v4/";
    console.info(`Отправка запроса на сервер...`)
    console.time('mathfetch');
    // запрос на сервер с обработкой ошибок
    return fetch(url, requestOptions)
    .then(async response => {
        if (!response.ok) {response.json().
            then(item=>console.error(item.error));
        throw new Error(`Ошибка сервера: ${response.status}`);}
        console.info(`Время получения запроса:`)
        console.timeEnd("mathfetch");
        return response.json();
    })
    .then(data => {
        data.input = expressionString;
        console.log("Результат вычисления:", data);
        return data; })
    .catch(error =>
        {console.error("Произошла ошибка при запросе:",
        error.message);});
}

```

Рисунок 24 – Функция отправки запроса на сервер

В этот запрос будет передаваться массив данных следующего вида:

```
// набор выражений и их результат
let unitTestSet = new Set([
    new Map([["expression","2 + 2 * 2"],["function",(math.sum(2, math.multiply(2,2)))]]),
    new Map([["expression","(2 + 2) * 2"],["function",(math.multiply(2, math.sum(2, 2)))]]),
    new Map([["expression", "1+1+1+1+1"],["function", math.sum(1, math.sum(1, math.sum(1, math.sum(1, 1))))]]),
    new Map([["expression", "0*0"],["function", math.multiply(0,0)]]),
    new Map([["expression", "1*0"],["function", math.multiply(1,0)]]),
    new Map([["expression", "0+0"], ["function", math.sum(0,0)]]),
    new Map([["expression", "0+1"], ["function",math.sum(0,1)]]),
    new Map([["expression","-9+1"], ["function",math.sum(-9,1)]]),
    new Map([["expression","-9+(-1)"], ["function",math.sum(-9,-1)]]),
    new Map([["expression","-6*(-1)"], ["function",math.multiply(-6,-1)]]),
    new Map([["expression","6*(-1)"], ["function",math.multiply(6,-1)]]),
    new Map([["expression","0!"], ["function",math.factorial(0)]]),
    new Map([["expression","1!"], ["function",math.factorial(1)]]),
    new Map([["expression","5!"], ["function",math.factorial(5)]])
]);

```

Рисунок 25 – Набор данных для модульного тестирования

В данном наборе данных нас интересуют два имеющихся поля:

- **Expression** – математическое выражение в виде строки
- **Function** – метод вычисления выражения на основе класса нашего локального Mathematical – фактически это результат вычислений, высчитываемый в момент обращения к переменной
- В последствии к этому набору добавляется поле **Result**, в него записывались результаты вычислений, просчитанных на стороне API

Все эти данные используются для отображения результата на странице.

После получения ответов на запрос, в интерфейсе создаются блоки, в которые записываются: исходное выражение, результат вычисления локальным классом, результат вычисления запроса, а также ответ в виде «Да\Нет» на вопрос «Совпадают ли результаты?». (см. Рисунок 27)

Результаты работы функционала методов в интерфейсе выглядят следующим образом:

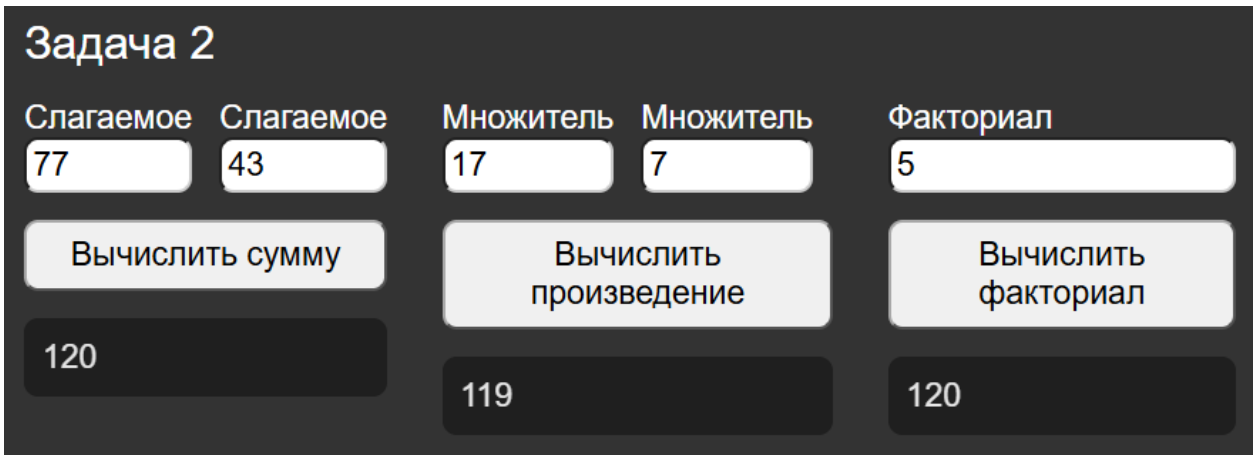


Рисунок 27 – Результаты работы методов

Результаты модульных тестов.

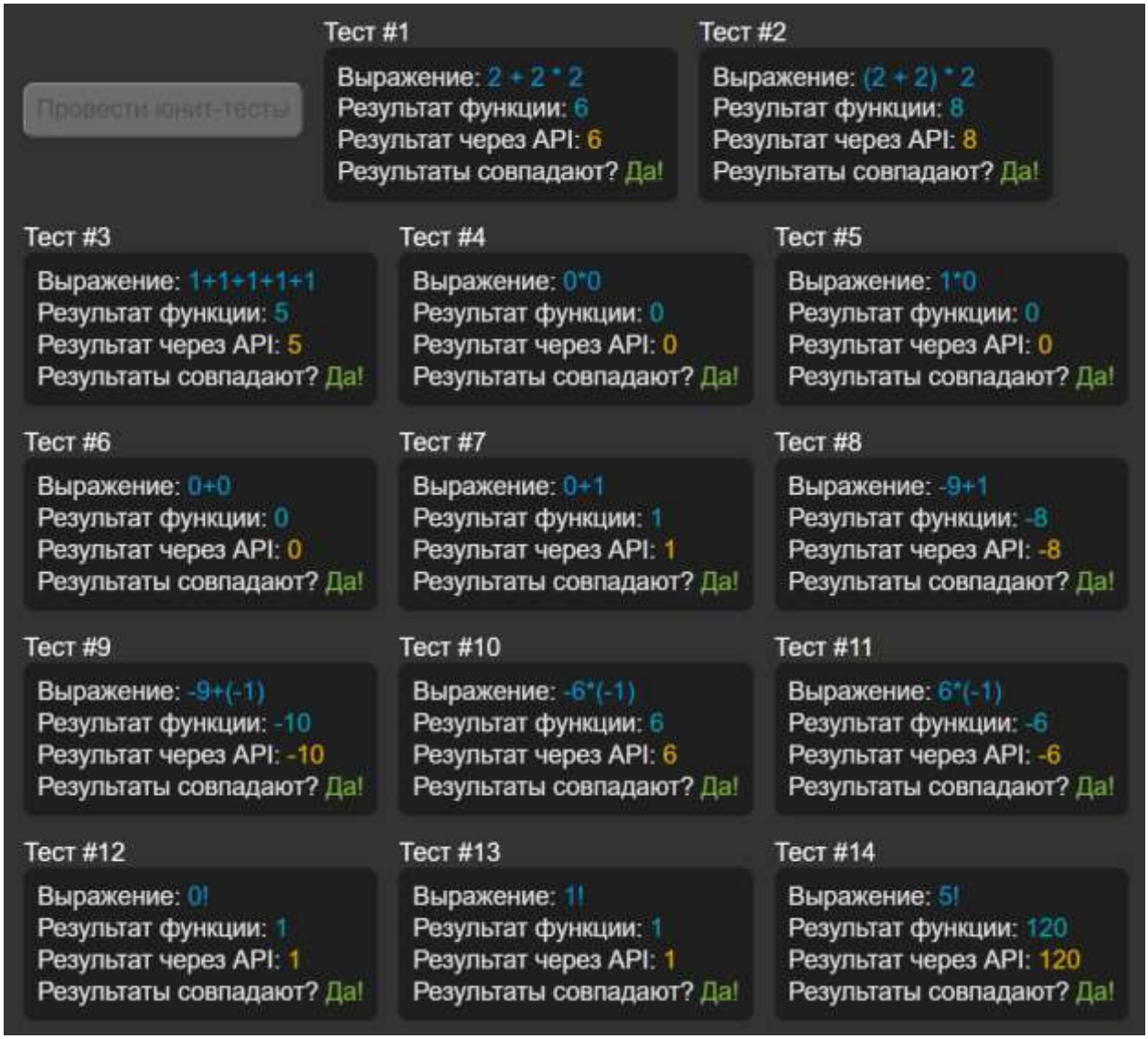


Рисунок 26 – Результаты тестов

Предотвращение ошибок

Одна из основных ошибок – передача «не числа» нашим функциям, занимающимся операциями над числами – была предотвращена на этапе интерфейса. Тег <input> в HTML это поле, позволяющее пользователям вписывать в него значения, и получать их с помощью JavaScript. Он имеет встроенный атрибут type=”number” который априори отрезают любые попытки написать не числа (допускается лишь написание отрицательных

чисел, если min имеет достаточно маленькое значение, чтобы в его рамки вписывались эти самые отрицательные числа).

В случае, если не передано вообще никакое выдаётся ошибка «Значение не определено» (см. Рисунок 27)

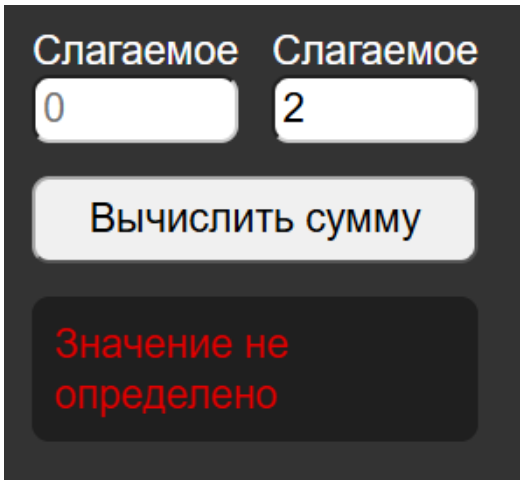


Рисунок 27 – Ошибка «Значение не определено»

Если же значение передаётся слишком большое и слишком маленькое (больше атрибута max или меньше атрибута min), то в «консоль» выводятся результаты следующего вида:

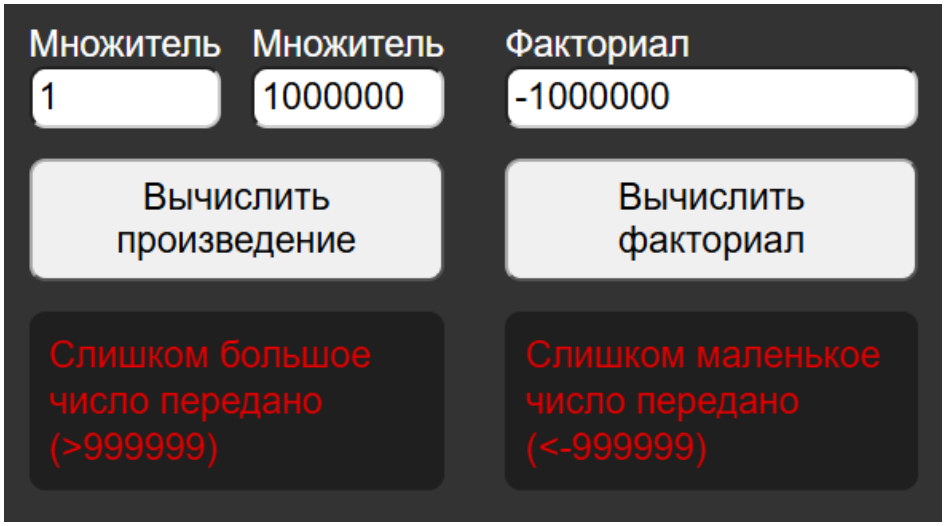


Рисунок 28 – Ошибка за допустимыми границами

Вывод

В ходе лабораторной работы был разработан модуль тестирования в рамках программного решения класса, занимающегося вычислением математических операций. В ходе Unit-тестирования было выявлено, что методы работают исправно и не вызывают ошибок. В интерфейсе было разработано предотвращение всех потенциально возникающих ошибок.

Таким образом были получены навыки автоматизации модульного тестирования средствами среды разработки.

Ссылка на проект на Github

Ссылка на исходный код на Github